

第4章 图形观察与变换

请思考：

- 1, 为什么我们从不同位置或者角度观察同一个物体，会看到不一样的效果？
- 2, 为什么有的人画的画真实感和立体感强？哪怕只是用铅笔画的。
- 3, 为什么一座房子能缩小到一张照片上？

内容提要：

- 4.1 二维图形变换
- 4.2 三维图形变换

4.1 二维几何变换

4.1.1 平移变换

4.1.2 旋转变换

4.1.3 比例变换

4.1.4 对称变换

4.1.5 错切变换

4.1.6 复合变换

4.1.7 二维观察变换

在方向、尺寸和形状方面的变化是通过改变图形坐标描述的几何变换来完成的

可改变和管理各种图形的显示

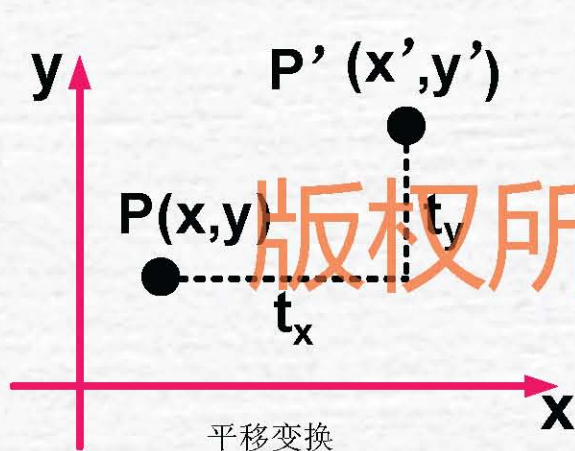
版权所有人：聂烜

OpenGL中对模型的几何变换也称为模型变换。
OpenGL中共提供了三类关于模型变换的函数，
即平移变换、旋转变换与比例变换。

它们的参数都包括了对象的 x 、 y 和 z 三个坐标，
可应用于三维物体。对于二维图形，我们将 z
坐标设为0，即假设变换是在 $z=0$ 的平面内进
行的。

4.1.1 平移变换

平移是指将物体沿直线路径从当前坐标位置移到另一个坐标位置的重定位过程。



设图形上点 $P(x, y)$, 将在 X 轴和 Y 轴方向分别移动 t_x 和 t_y , 结果生成新的点

$P'(x', y')$, 如图所示, 则有

$$x' = x + t_x \quad y' = y + t_y$$

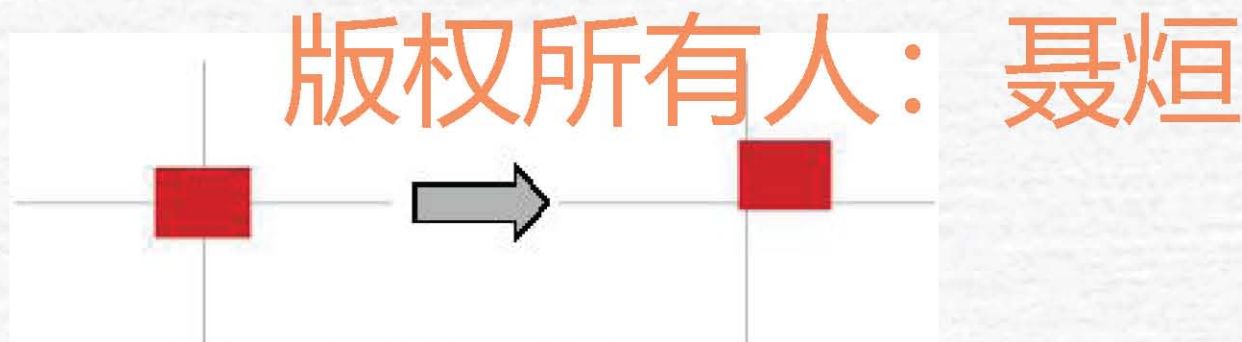
其中 t_x, t_y 称为点在 X 轴和 Y 轴上的位移。

如果 t_x 或 t_y 大于零, 则点向右或向上移动; 如果 t_x 或 t_y 小于零, 则点向左或向下移动。

4.1.1 平移变换（续）

为了方便几何变换的表示，通常用齐次坐标和矩阵形式对其进行表达：

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$



平移是一种不产生变形而移动物体的刚体变换（**Rigid-body transformation**），即物体上的每一点移动相同量的坐标。

OpenGL 平移函数的原型为：

- `void glTranslated(GLdouble x, GLdouble y, GLdouble z);`
- `void glTranslatef(GLfloat x, GLfloat y, GLfloat z);`

其中， x, y, z 分别为在 x, y 和 z 方向上的平移量，对于二维图形 $z=0$ 。

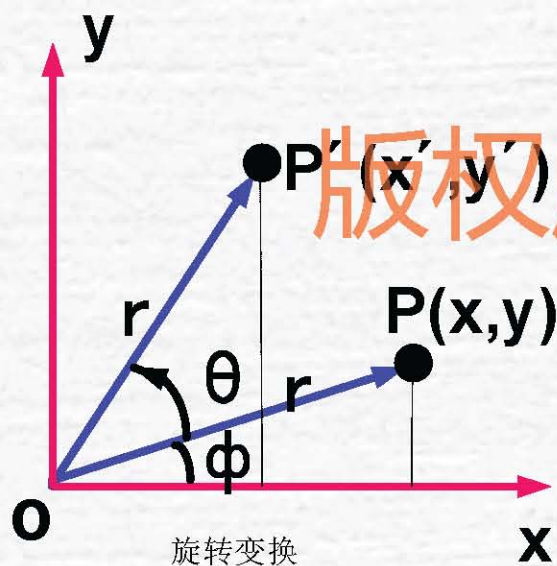
`glTranslate* (x, y, 0)` 所生成的等价二维变换矩阵为：

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x & y & 1 \end{bmatrix}$$

4.1.2 旋转变换

二维物体**旋转**是指将物体沿某个定点转动某个角度的重定位过程。

设有图形上点 $P(x,y)$ ，将其绕原点旋转变换 α 角度(假设按逆时针旋转为正角)，结果生成的新的点坐标 $P'(x',y')$ 。



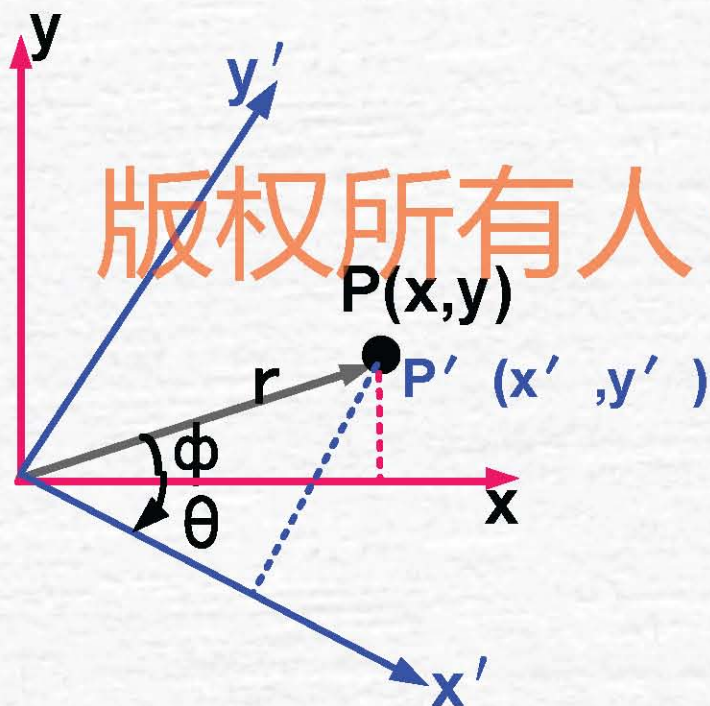
$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

其中 θ 为点绕原点旋转的角度(逆时针为正，顺时针为负)。

4.1.2 旋转变换 (续)

问题：将点 P 绕原点做逆时针旋转 θ 角度的变换看作将坐标系绕原点做顺时针旋转 θ 角度的等价变换？



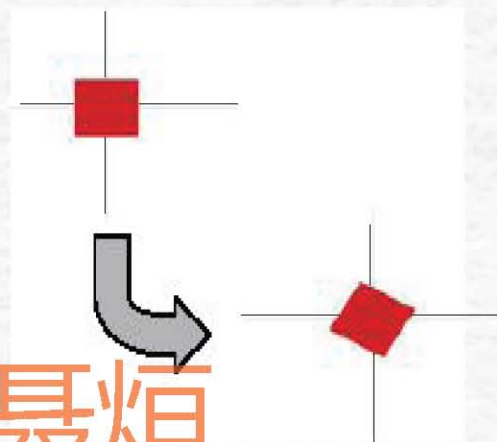
版权所有人：聂烜

4.1.2 旋转变换 (续)

用齐次坐标和矩阵形式可表示为:

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

版权所有人：聂烜



与平移变换一样，旋转变换也是一种不产生变形地移动物体的刚体变换。物体上所有的点旋转相同的角度。

OpenGL 旋转变换函数的原型为:

- `void glRotated(GLdouble angle, GLdouble x, GLdouble y, GLdouble z);`
- `void glRotatef(GLdouble angle, GLfloat x, GLfloat y, GLfloat z);`

其中, `angle` 指定逆时针方向旋转的角度, 范围 0° – 360° ,
`x`, `y`, `z` 与原点相连, 产生旋转轴。

对于二维图形, 由于只能在 xy 平面内旋转。因此只能绕 z 轴旋转, 即相当于在二维平面内绕原点旋转。这时参数可取为:
`x=0`, `y=0`, `z=1`。

函数 `glRotate* (θ, 0, 0, 1)` 所生成的等价二维变换矩阵为:

$$\mathbf{T} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

4.1.3 比例变换

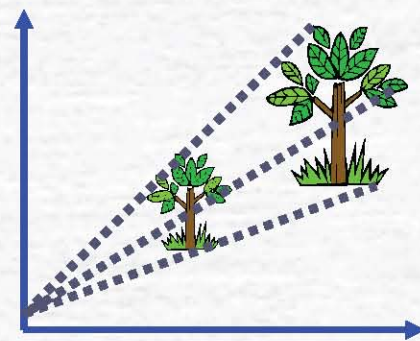
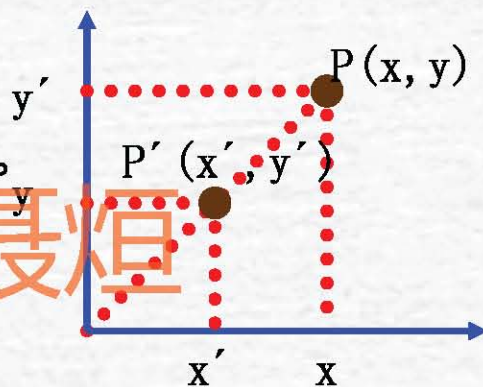
对于初始点 $P(x, y)$ 的比例变换是指相对于坐标原点，其坐标在 X 轴和 Y 轴方向分别作 s_x 倍和 s_y 倍的缩放，得到变换后的点坐标 $P'(x', y')$ ，如图所示，有 $x' = s_x \cdot x$ ，

$$y' = s_y \cdot y$$

其中 s_x ， s_y 称为点在 X 轴和 Y 轴上的缩放倍数。

用齐次坐标和矩阵形式可表示为：

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



4.1.3 比例变换（续）

对于上述缩放倍数有 s_x 和 s_y 以下结论

- 如果 $|s_x|$ 或 $|s_y|$ 大于1，则表示图形在x轴方向或y轴方向放大；
- 如果 $|s_x|$ 或 $|s_y|$ 小于1，则表示图形在x轴方向或y轴方向缩小；
- 如果 $|s_x|$ 或 $|s_y|$ 等于1，则表示图形在x轴方向或y轴方向不变；
- 如果 $s_x = 1$ ， $s_y = -1$ ，则等价于图形关于x轴作对称变换；
- 如果 $s_x = -1$ ， $s_y = 1$ ，则等价于图形关于y轴作对称变换。

OpenGL 比例变换函数的原型为：

- `void glScaled(GLdouble x, GLdouble y, GLdouble z);`
- `void glScalef(GLfloat x, GLfloat y, GLfloat z);`

其中， x, y, z 分别表示对象沿三个坐标轴缩放的比例因子。
对于二维图形，其 z 坐标值等于 0。

`glScale(x, y, 0)` 所生成的二维变换矩阵为：

版权所有人：聂烜

$$T = \begin{bmatrix} x & 0 & 0 \\ 0 & y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

`glScale(1, -1, 0)`

`glScale(-1, -1, 0)`

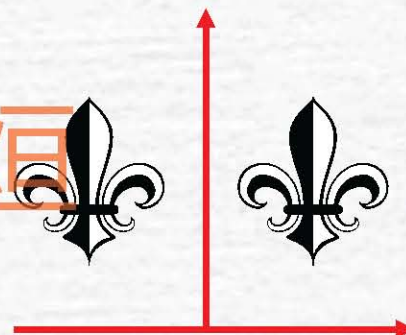
上述两函数产生的变换矩阵为？

4.1.4 对称变换

对称变换也称为反射变换，相对于反射轴的对称变换是通过将物体绕反射轴旋转**180°**而生成的。它的基本变换包括对坐标轴、原点和**45°**线的变换。

✓关于**x**轴的对称变换，是保持**x**值不变，“翻动”**y**坐标位置而得到的。这时的变换矩阵：

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



4.1.4 对称变换 (续)

✓ 相对于y轴的对称变换即保持y坐标不变，而对x坐标取相反数而得到，这时的变换矩阵：

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

✓ 关于原点对称的变换矩阵？

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

版权所有人：聂烜

4.1.4 对称变换 (续)

✓关于 $\mathbf{y}=\mathbf{x}$ 对称的变换矩阵?

$$\begin{bmatrix} \mathbf{x}' & \mathbf{y}' & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{x} & \mathbf{y} & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

✓关于 $\mathbf{y}=-\mathbf{x}$ 对称的变换矩阵?

$$\begin{bmatrix} \mathbf{x}' & \mathbf{y}' & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{x} & \mathbf{y} & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

版权所有人：聂烜

4.1.5 错切变换

错切(shear)变换也称为错位或错移变换，变换结果将使目标图形失真。

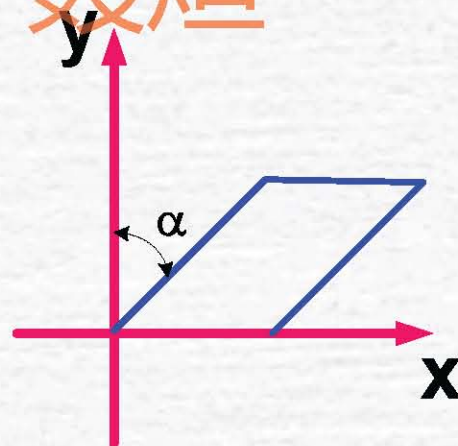
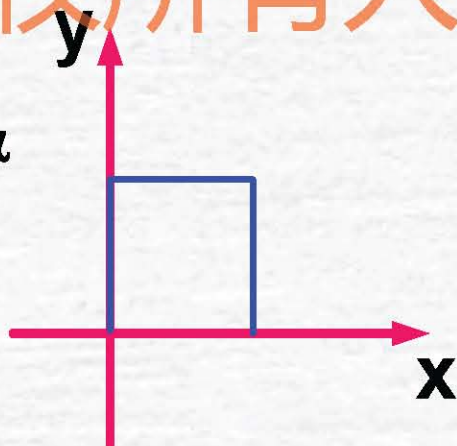
✓ 沿x方向的错切

如下图，变换的结果是使图形的y坐标不变，而x坐标有一个增量，图形沿x轴产生错切变形。

$$x' = x + cy$$

$$y' = y$$

其中， $c = \tan \alpha$



版权所有人：聂烜

4.1.5 错切变换 (续)

✓ 沿x方向的错切

当 $c > 0$ 时, 如 $y > 0$, 沿 $+x$ 方向错切; $y < 0$, 沿 $-x$ 方向错切。
当 $c < 0$ 时, 如 $y > 0$, 沿 $-x$ 方向错切; $y < 0$, 沿 $+x$ 方向错切。

其所对应的齐次坐标变换矩阵为:

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

版权所有人: 聂烜

4.1.5 错切变换 (续)

✓ 沿y方向的错切

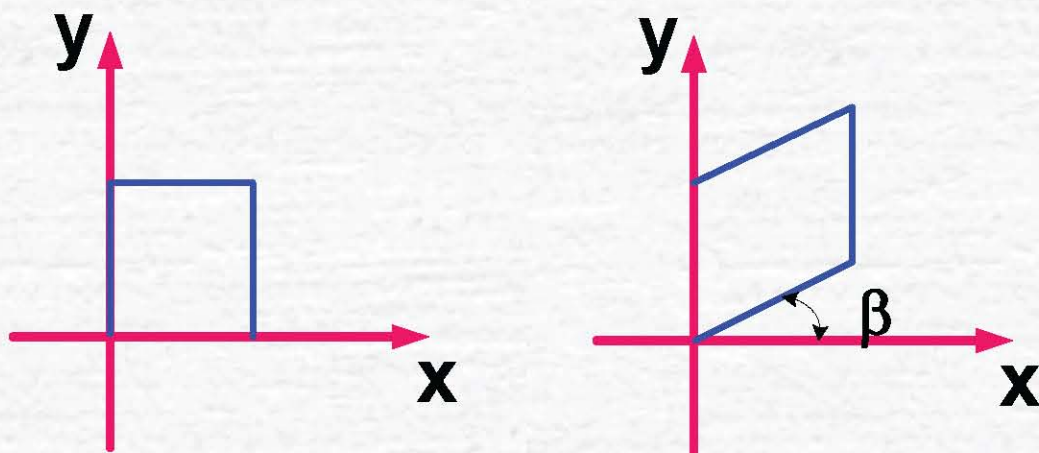
如下图，变换的结果是使图形的x坐标不变，而y坐标有一个增量，图形沿y轴产生错切变形。

$$x' = x$$

$$y' = bx + y$$

其中， $b = \tan \beta$

版权所有人：聂烜



4.1.5 错切变换 (续)

✓ 沿y方向的错切

当 **$b > 0$** 时, 如 **$x > 0$** , 沿 **$+y$** 方向错切; **$x < 0$** , 沿 **$-y$** 方向错切。

当 **$b < 0$** 时, 如 **$x > 0$** , 沿 **$-y$** 方向错切; **$x < 0$** , 沿 **$+y$** 方向错切。

其所对应的齐次坐标变换矩阵为:

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & b & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

版权所有人：聂烜

4.1.6 复合变换

实际上，一般的图形变换更多的是复合变换，这类变换必须按一定的顺序进行多次基本的几何变换才能实现，则复合变换矩阵 T 也可由一系列基本几何变换矩阵 T_i 的乘积来表示，即： $T = T_1 \cdot T_2 \cdots T_n$

下面将举例说明复合变换的方法。

复合变换举例

例1:平面图形绕任意点 $P(x_p, y_p)$ 旋转 θ 角，可以经过以下几个步骤实现：

- 1) 将旋转中心移到原点，变换矩阵为：

$$T_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_p & -y_p & 1 \end{bmatrix}$$

- 2) 将图形绕原点旋转 θ 角，变换矩阵为：

$$T_2 = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- 3) 将旋转中心平移回原来的位置，变换矩阵为：

$$T_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_p & y_p & 1 \end{bmatrix}$$

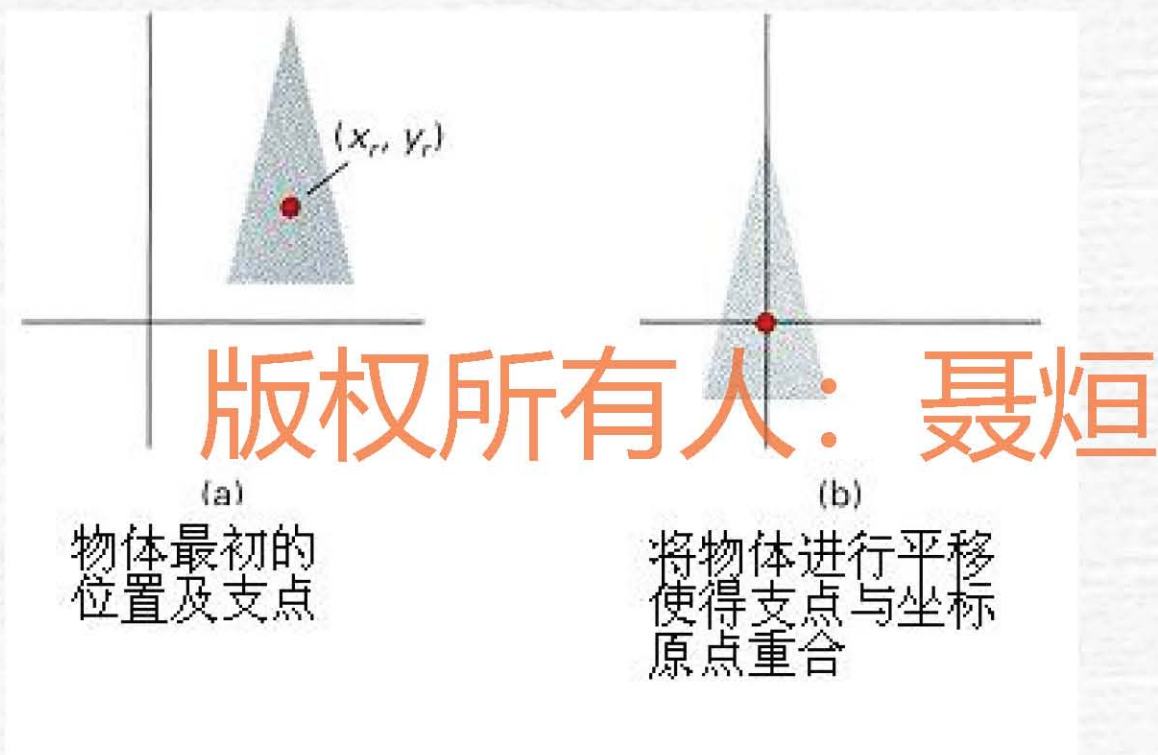
复合变换举例

于是绕任意点 P 旋转的变换矩阵为：

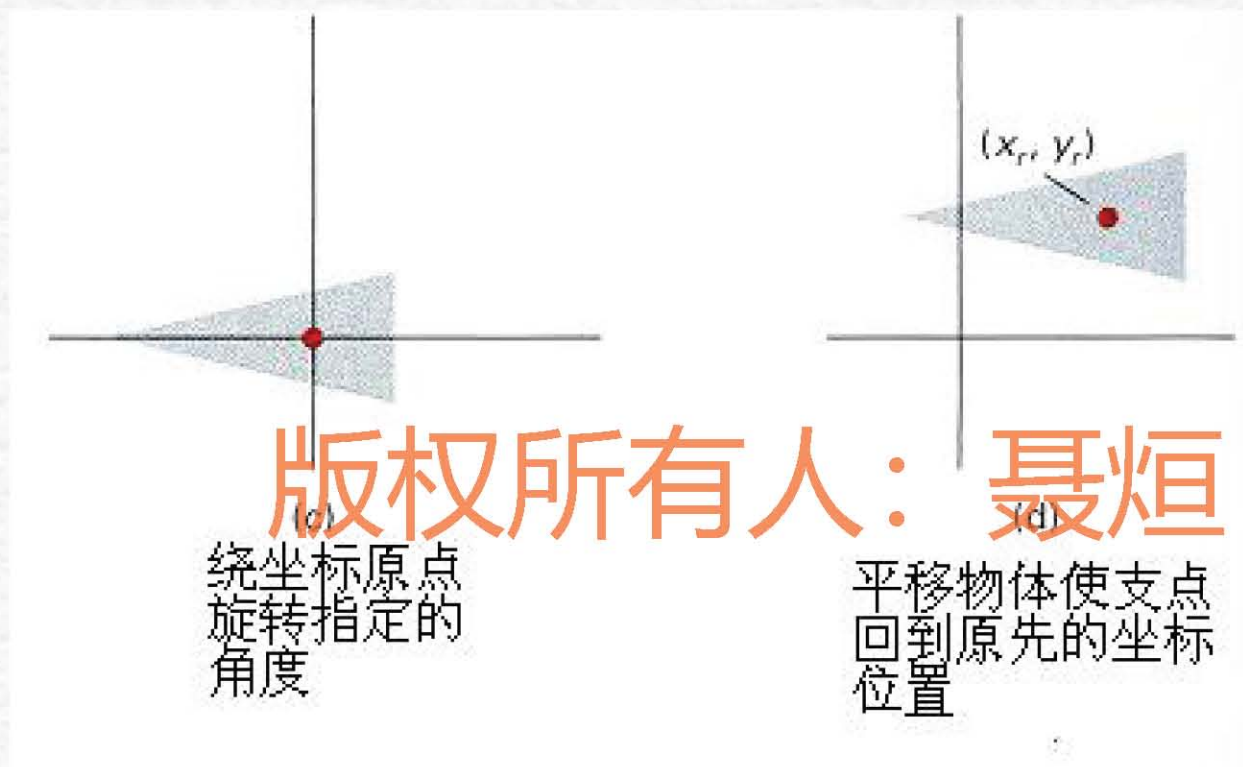
$$T = T_1 \cdot T_2 \cdot T_3$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_p & -y_p & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_p & y_p & 1 \end{bmatrix}$$

平面图形绕任意点旋转举例：



平面图形绕任意点旋转举例（续）：



复合变换举例

例2:平面图形相对于任意点 $P(x_p, y_p)$ 作比例变换可通过以下几个步骤来完成:

1) 将 p 点平移到坐标原点, 变换矩阵为:

$$T_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_p & -y_p & 1 \end{bmatrix}$$

2) 关于原点作比例变换, 变换矩阵为:

$$T_2 = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3) 将旋转中心平移回原来的位置, 变换矩阵为:

$$T_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_p & y_p & 1 \end{bmatrix}$$

复合变换举例

相对于任意点作比例变换的变换矩阵为：

$$T = T_1 \cdot T_2 \cdot T_3$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_p & -y_p & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_p & y_p & 1 \end{bmatrix}$$

4.1.6 二维观察变换

图形的**二维观察**是通过指定一个在图形中要**显示的部分**以及在**显示器显示位置**，并执行从世界坐标系到设备坐标系的**图形变换**及删除位于显示区域范围以外的图形部分而实现的。

- 版权所有人：聂烜
1. 二维图形**坐标系统**
 2. 二维观察流程
 3. 窗口到视区的变换
 4. 二维图形裁剪

4.1.6 二维观察变换（续）

1. 二维图形坐标系

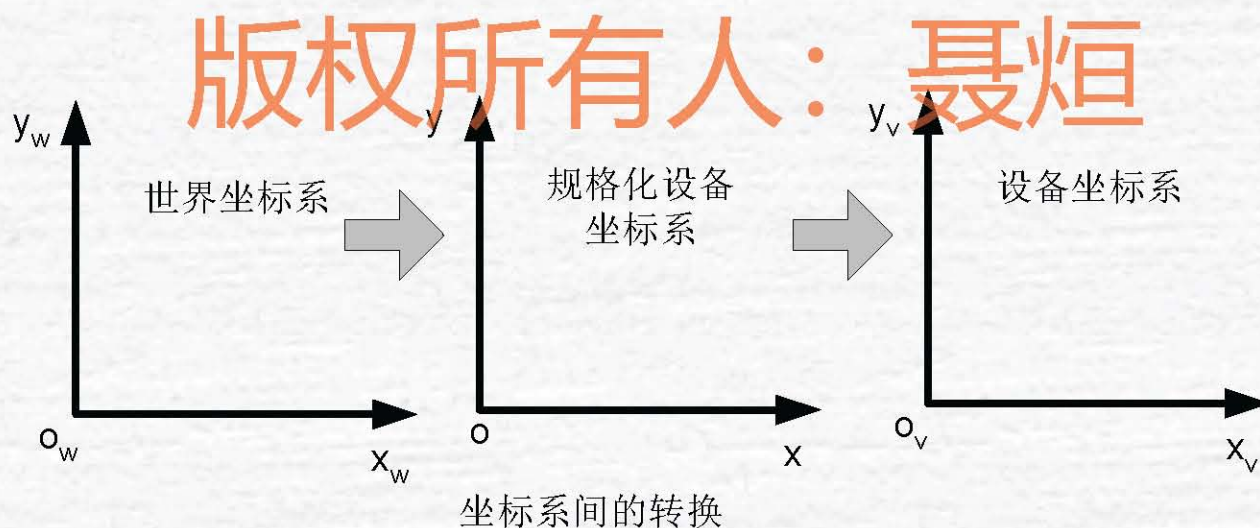
在计算机图形学中，为了通过显示设备来考察几何物体的特性，引入了一系列用于显示输出的坐标系，而图形的显示过程就可以看成对象模型是在不同坐标系间的映射。

✓ **世界坐标系**：用户用来定义图形的坐标系称为世界坐标系，也称为用户坐标系；它的定义域是实数域，二维的世界坐标系通常用 $O_w x_w y_w$ 表示。

✓ **设备坐标系**：在显示器和绘图仪等图形的输出设备上也有一个自身的坐标系，该坐标系称为设备坐标系或物理坐标系，简称为 **DC**。设备坐标系是一个二维坐标系，对于显示器而言它的度量单位是像素。其定义域？

✓**规格化设备坐标系**：它独立于设备坐标系和世界坐标系，又可容易地转变成设备坐标系的一个坐标系，是一个中间坐标系。在该坐标系中，**x**和**y**轴的坐标被规格化为**0~1**间的量。

当我们输出图形时，首先将世界坐标系转换为与设备无关的设备坐标系，再由规格化设备坐标系转换成具体的设备坐标。



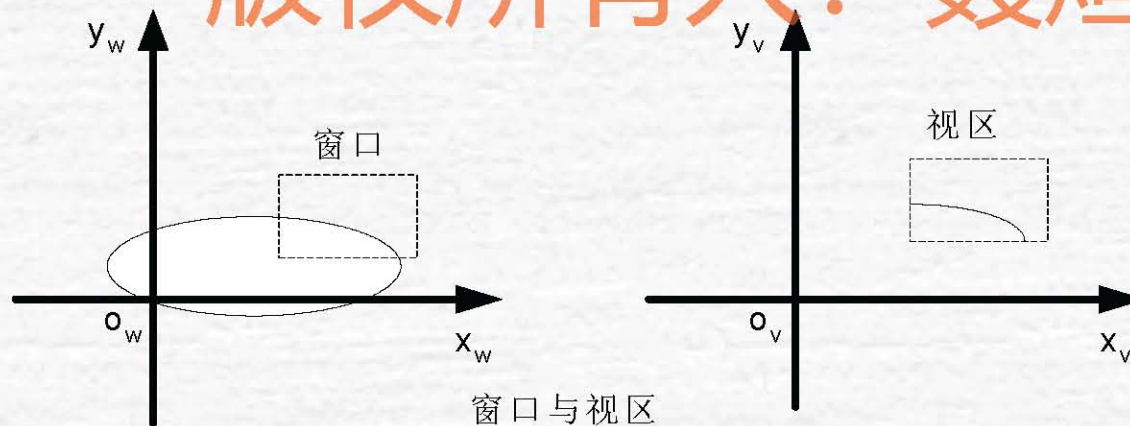
4.1.6 二维观察变换 (续)

2. 二维观察流程

在世界坐标系中要显示的区域称为**窗口**；窗口映射到显示设备上的坐标区域称为**视区**。标准窗口与视区一般都采用矩形，其各边分别与坐标平行。

窗口与视区的区别？ (...)

版权所有人：聂烜



4.1.6 二维观察变换（续）

窗口与视区

- ✓ **窗口** 定义了我们显示的内容。
- ✓ **视区** 决定在显示设备上的显示位置，我们可以在输出设备的不同位置观察物体；也可以通过改变视区的尺寸来改变显示对象的尺寸和位置。
- ✓ **窗口和视区** 分别处在不同的坐标系内，它们所用的长度单位及大小位置等不同。因此要将窗口内的内容在视区中显示出来，必须经过从窗口到视区的变换处理，这种变换即称为**观察变换**。

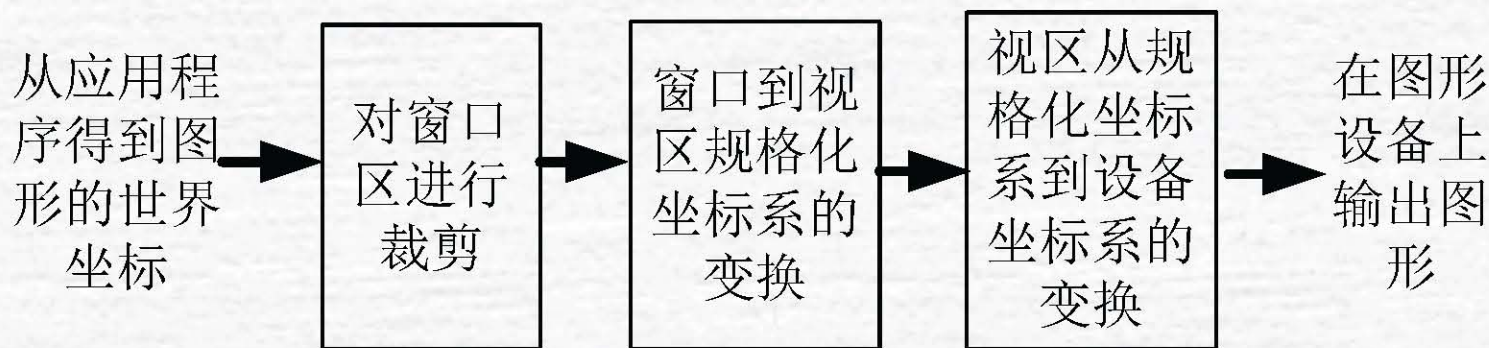
4.1.6 二维观察变换（续）

二维观察变换

- 1) 在世界坐标系中生成图形
- 2) 裁剪，得到要显示的内容
- 3) 窗口到视区的变换

窗口——规格化设备坐标系中的视区——设备坐标系中的视区

- 4) 显示图形

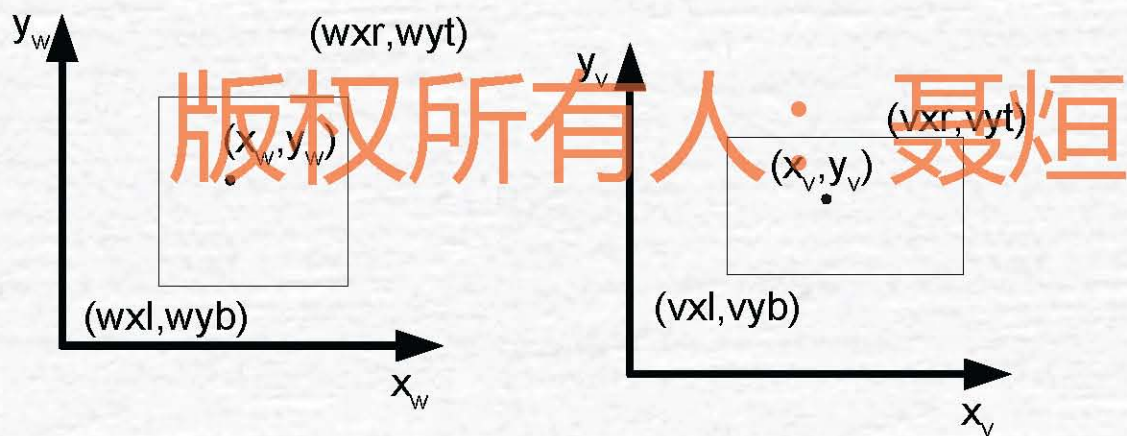


二维观察的变换流程

4.1.6 二维观察变换 (续)

3. 窗口到视区的变换

解决了如何将窗口内的点 (x_w, y_w) 变换到视区内的点坐标 (x_v, y_v) 的问题。



从窗口到视区的变换

4.1.6 二维观察变换（续）

4. 二维图形裁剪

由于观察变换中仅需要显示出窗口内的区域，而窗口外的区域就被去掉。因此在进行观察变换之前，先需要确定图形中的哪些部分位于窗口内，哪些部分位于窗口之外，以便于对窗口内的图形进行显示。**识别图形在指定区域内或指定区域外的过程称为裁剪。**

在OpenGL中，二维观察变换实现函数为：

✓ `gluOrtho2D(0.0, 0.0, (GLdouble)w, (GLdouble)h)`
该函数定义窗口的大小。

✓ `glViewport(0, 0, (GLsizei)w, (GLsizei)h)`
该函数定义视口的大小。

举例

4.2 三维图形变换

4.2.1 三维几何变换

4.2.2 投影变换

4.2.3 三维观察变换

4.2.4 OpenGL中的三维图形变换

版权所有：聂烜

4.2.1 三维几何变换

三维几何变换可以看作是二维几何变换的扩展，有关二维图形几何变换的方法大部分都可应用于三维图形。

与二维图形一样，我们在介绍三维图形几何变换时也采用齐次坐标来进行描述。

(x, y, z) 用齐次坐标表示为 $(x, y, z, 1)$ 。如果用坐标来表示三维空间中变换前的点，用来表示变换后的结果，则其变换方程为：

$$[x' \ y' \ z' \ 1] = [x \ y \ z \ 1] \cdot T$$

x, y, z 轴关系符合右手坐标系

右手坐标系:



4.2.1 三维几何变换（续）

1. 平移变换

若三维物体沿 x 、 y 、 z 方向上移动一个位置，其大小和形状均保持不变，则称为平移变换。平移变换的矩阵表示为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$$

即满足：

$$\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \\ z' &= z + t_z \end{aligned}$$

4.2.1 三维几何变换（续）

2. 比例变换

相对于坐标原点的比例变换的矩阵表示为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

即满足：

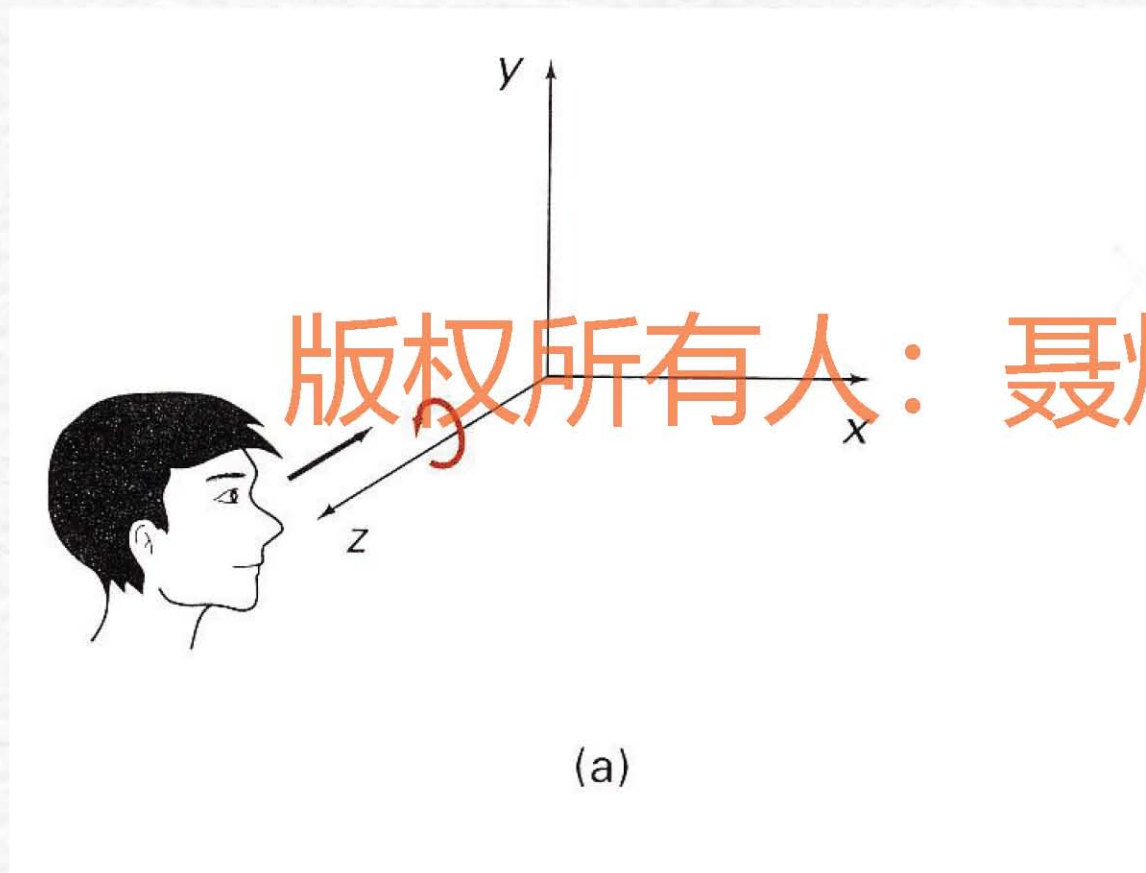
$$x' = s_x \cdot x$$

$$y' = s_y \cdot y$$

$$z' = s_z \cdot z$$

3. 旋转变换

1) 绕Z轴旋转

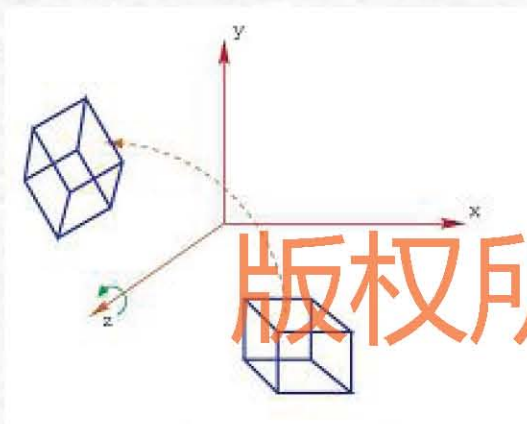


Z: X → Y

4.2.1 三维几何变换（续）

绕z轴旋转

三维物体绕z轴旋转，则物体上各点的z坐标保持不变。**x,y**方向的坐标绕原点旋转。



$$x' = x \cos \theta - y \sin \theta$$

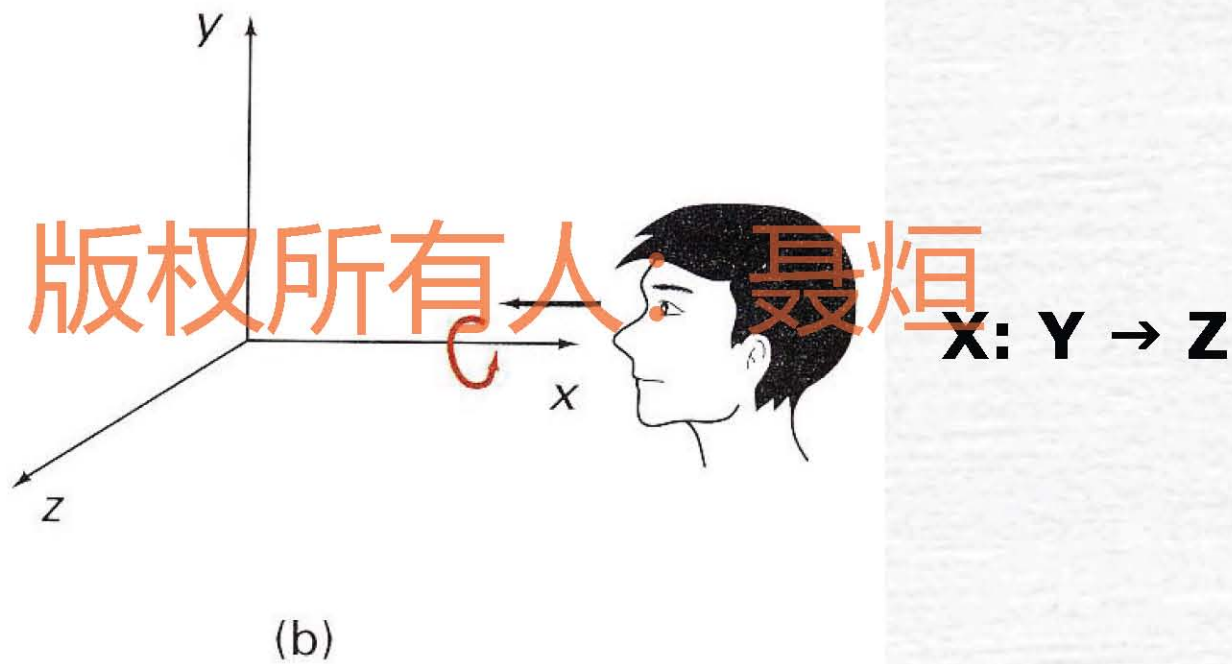
$$y' = x \sin \theta + y \cos \theta$$

版权所有人：聂烜

其对应的变换矩阵可表示为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -\sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

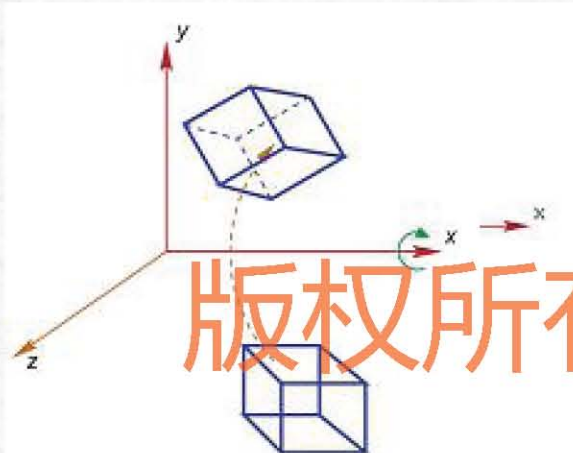
2) 绕x轴旋转



4.2.1 三维几何变换（续）

绕x轴旋转

三维物体绕x轴旋转，则物体上各点的x坐标保持不变。y,z方向的坐标绕原点旋转。



$$y' = y \cos \theta - z \sin \theta$$

$$z' = y \sin \theta + z \cos \theta$$

$$x' = x$$

版权所有人：聂烜

其对应的变换矩阵可表示为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3) 绕y轴旋转



(c)

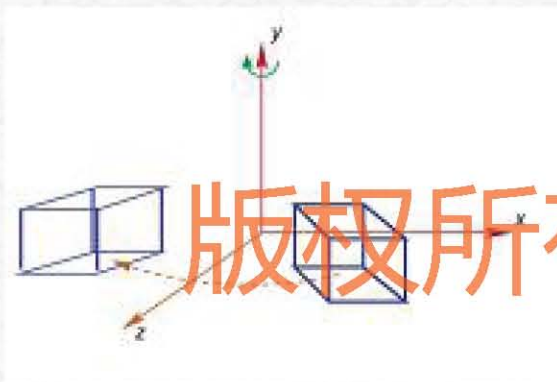
版权所有人: 聂烜

Y: $Z \rightarrow X$

4.2.1 三维几何变换（续）

绕y轴旋转

三维物体绕y轴旋转，则物体上各点的y坐标保持不变。x,z方向的坐标绕原点旋转。



$$z' = z \cos \theta - x \sin \theta$$

$$x' = z \sin \theta + x \cos \theta$$

$$y' = y$$

其对应的变换矩阵可表示为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.2.1 三维几何变换（续）

4. 对称变换

1) 关于坐标平面的对称变换

空间中一点关于**xoy**坐标平面的对称变换：点的**x**和**y**向的坐标保持不变，只改变**z**坐标的正负号，即：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} x & y & -z & 1 \end{bmatrix}$$

空间中一点关于**yoz**和**zox**坐标平面的对称变换的变换矩阵？

4.2.1 三维几何变换（续）

2) 关于坐标轴的对称变换

空间中一点关于x轴作对称变换时，则点的x坐标保持不变，只改变y和z方向的坐标的正负号，即：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} x & -y & -z & 1 \end{bmatrix}$$

空间中一点关于y轴和z轴对称变换的变换矩阵？

4.2.1 三维几何变换（续）

5. 切错变换

三维物体的某个面沿指定轴向移动属于三维错切。

1) 沿x轴方向的切错

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ d & 1 & 0 & 0 \\ g & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} x+dy+gz & y & z & 1 \end{bmatrix}$$

沿y轴和z轴方向的切错？

4.2.1 三维几何变换（续）

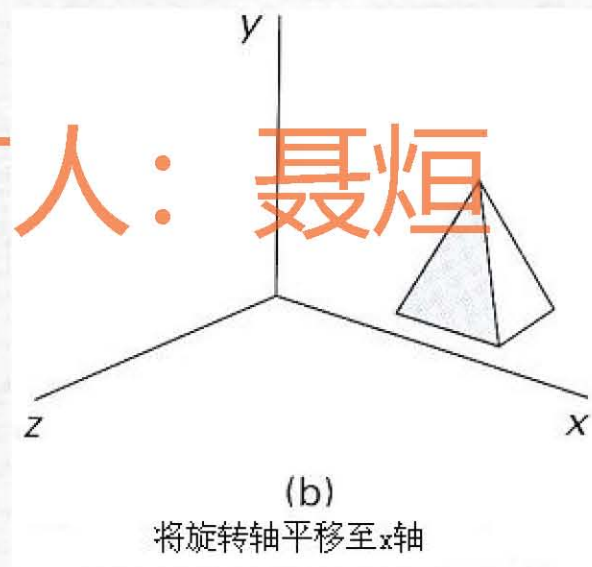
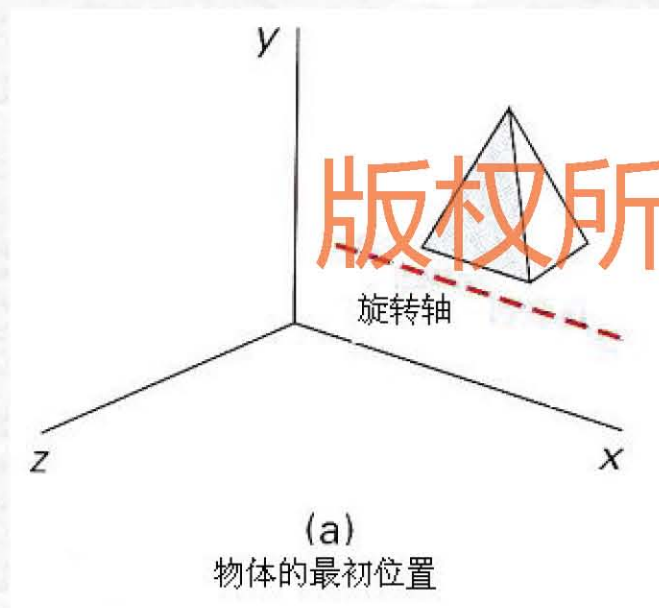
6. 三维复合变换

上述三维图形基本的几何变换都是针对原点或者坐标轴进行的，如果要针对任意一个参考点，或者任意一条直线来进行变换，就需要进行三维图形的复合变换。三维复合变换是指图形作一次以上的变换，变换结果是每次的变换矩阵相乘，可表示为：

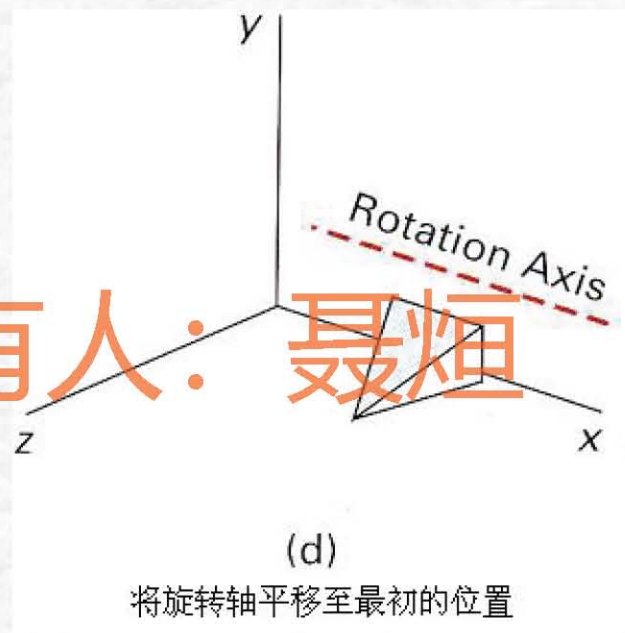
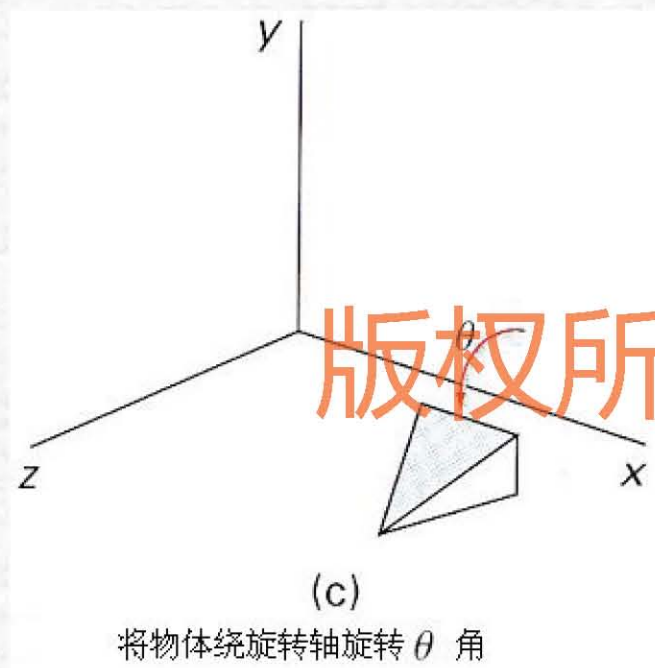
$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot (T_1 \cdot T_2 \cdots T_n)$$

4.2.1 三维几何变换（续）

例1： 三维物体绕平行于 x 轴的直线旋转 θ 角



4.2.1 三维几何变换（续）



4.2.2 投影变换

问题：

通常的图形输出设备如显示器、绘图仪等都是**二维**的。而我们在**三维世界坐标系**中对场景模型进行描述，要将三维物体在这些二维设备上显示，就要把三维坐标中的各点转化为二维平面坐标系中的点。

版权所有人：聂烜

解决方法：

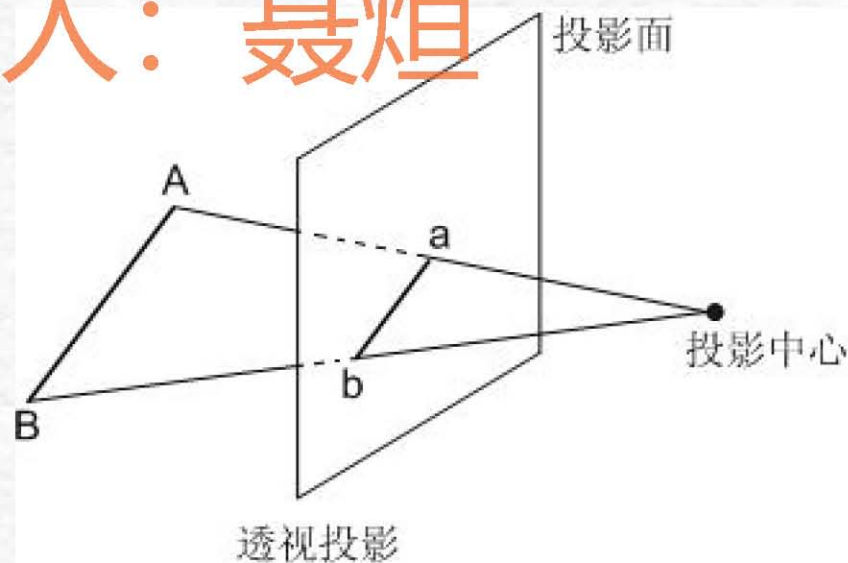
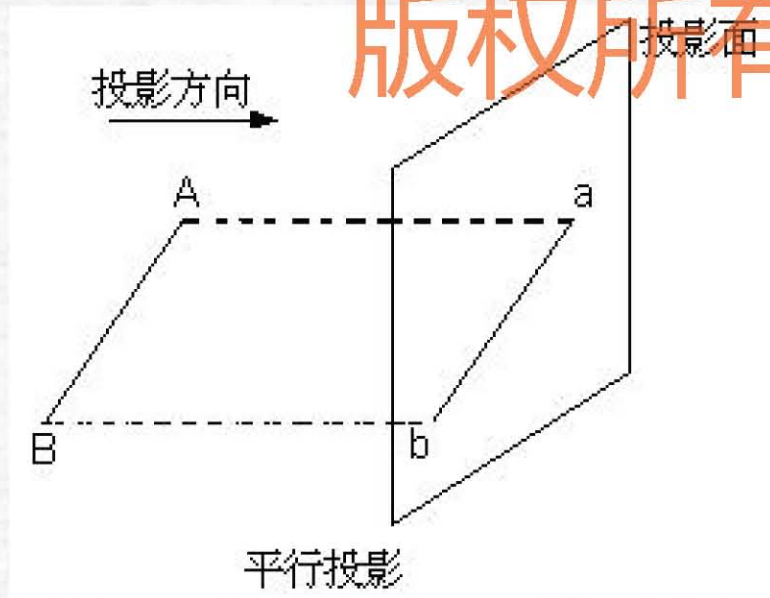
三维投影变换：把三维物体变为二维图形表示。

投影变换分为**平行投影**与**透视投影**两类。

4.2.2 投影变换(续)

在平行投影中，物体的坐标位置沿平行线变换到投影平面上。

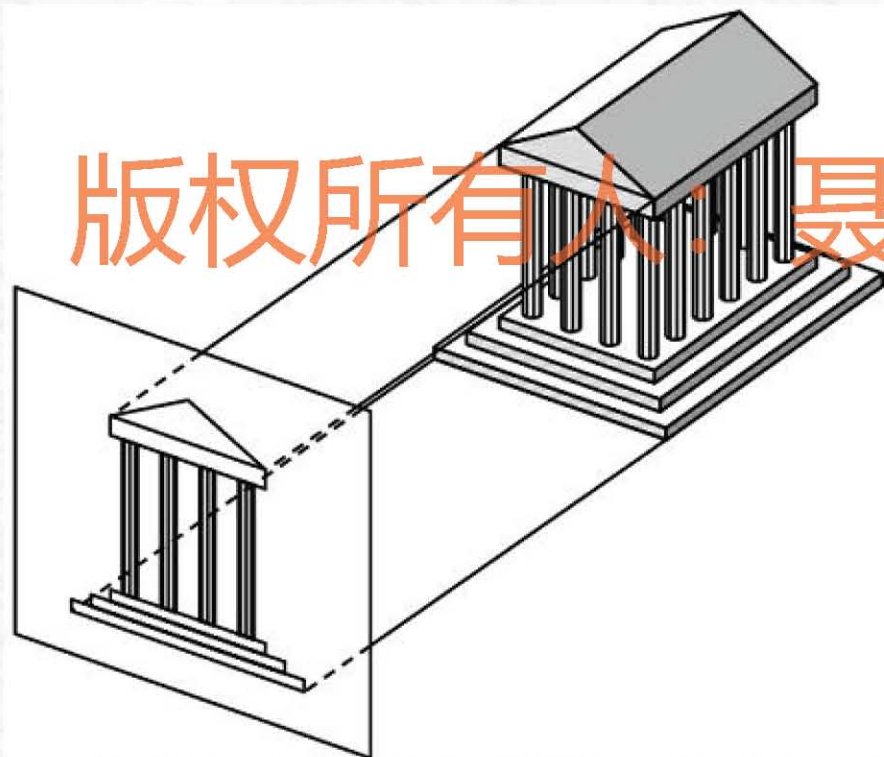
在透视投影中，定义投影平面后的一点为投影中心，将投影中心和三维物体上的各点的连线称为投影线。投影线与投影面的交点即称为各点的投影。



平行投影

平行投影的特点

- ✓ 平行投影保持物体的大小比例不变，物体各个面的精确视图可有平行投影得到，但无法给出三维物体的真实性表示。



例

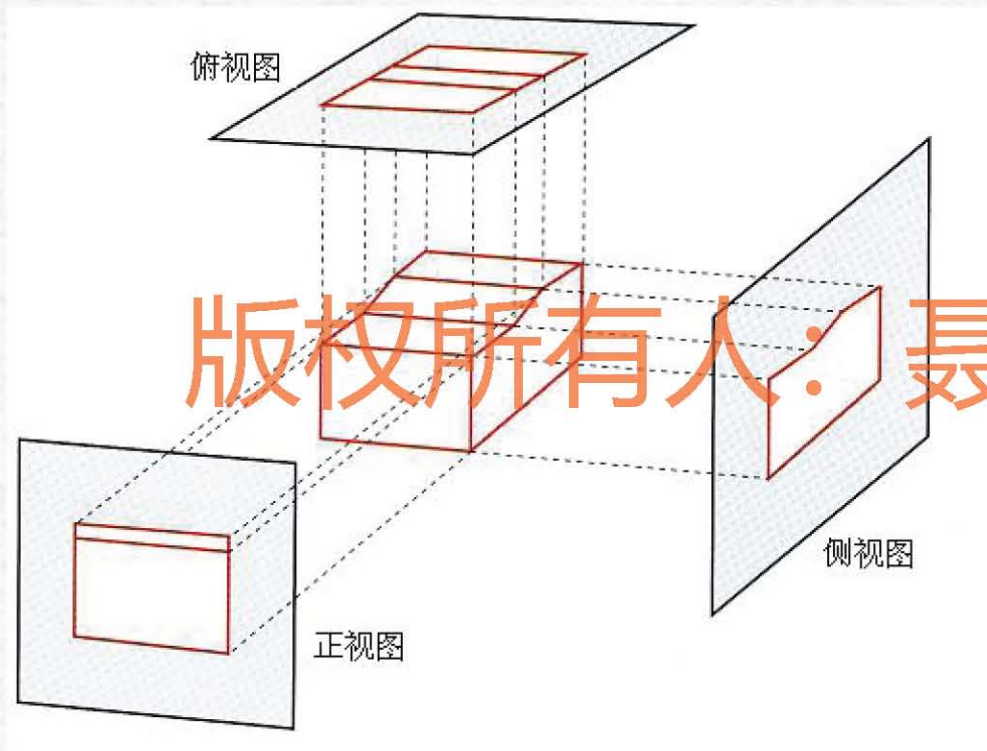
平行投影

✓ 根据平行投影与投影平面的间的夹角也可分为**正平行投影**与**斜平行投影**两类，当投影线垂直于投影平面时，得到的投影称为正平行投影；否则为斜平行投影。



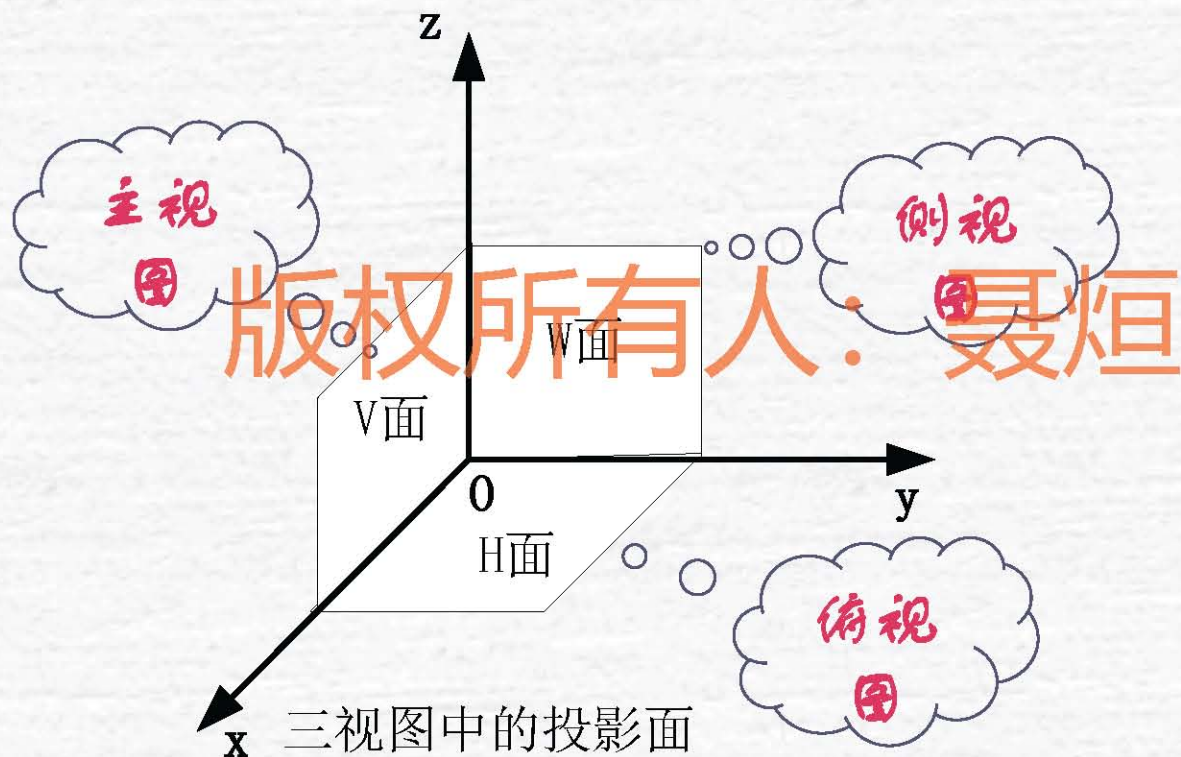
正平行投影

我们通常所说的三视图即正视图、俯视图与侧视图即属于正平行投影。



正平行投影

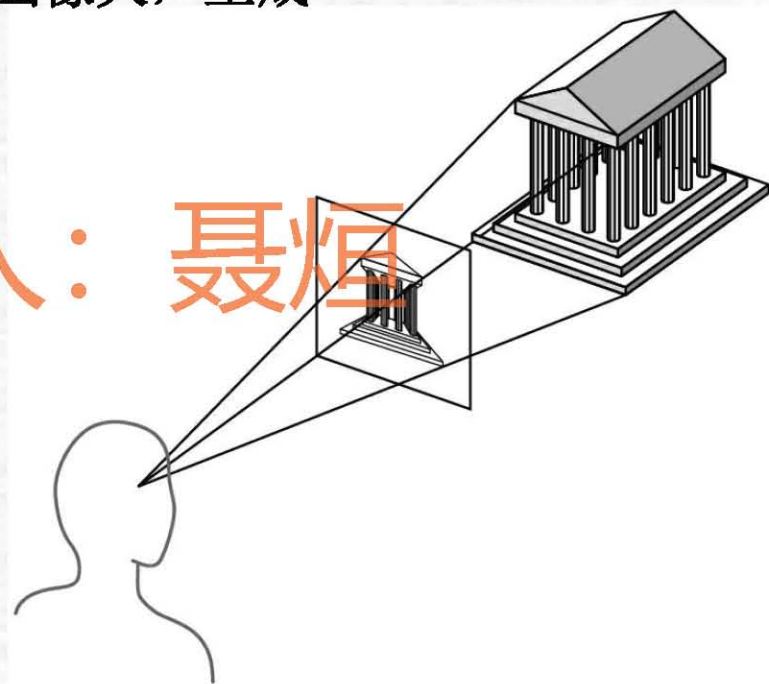
在工程上将三维坐标系 $Oxyz$ 的三个坐标平面分别分为V面(xOz 平面)、H面(xOy 平面)和W面(yOz 平面)，如下图：



透视投影

在平行投影中，物体投影的大小与物体距投影面的距离无关，与人的视觉成像不符。而在透视投影中，离投影面近的物体较远的物体生成的图像大，生成真实感图形。

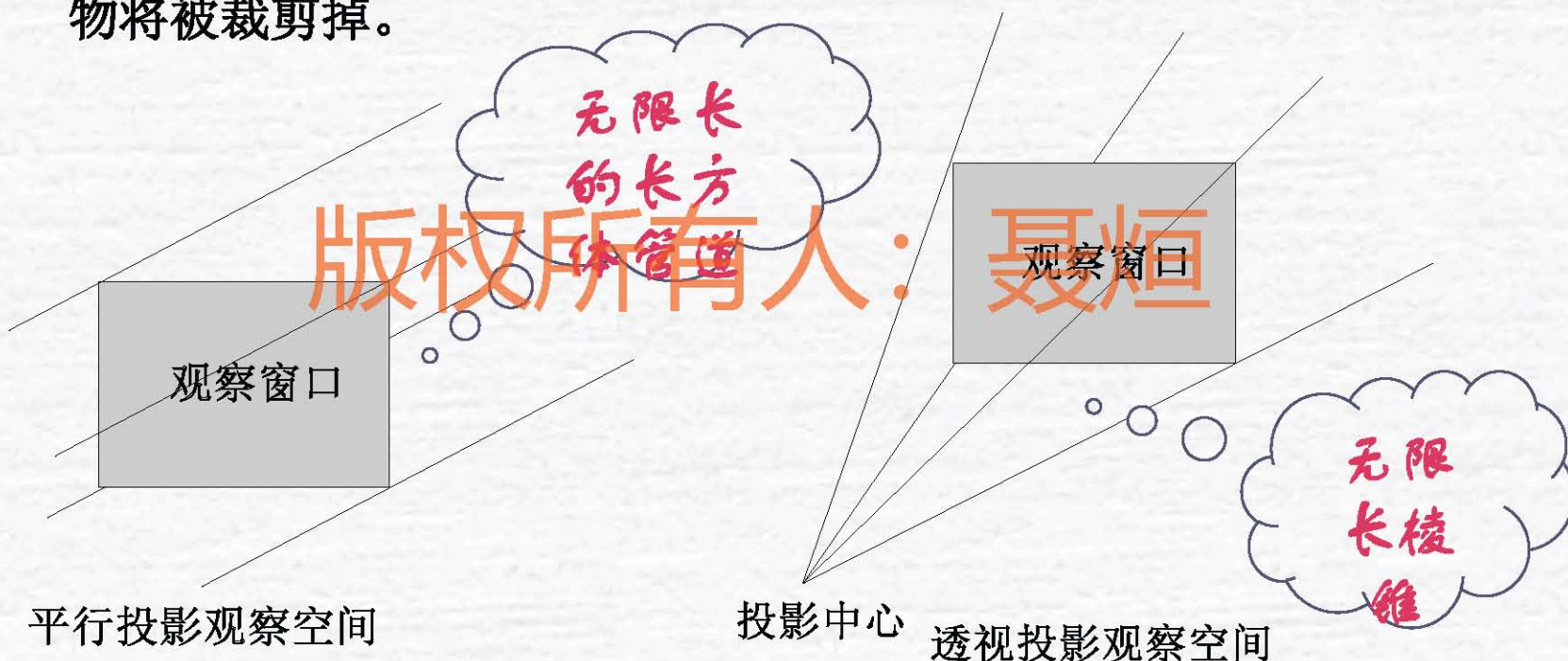
投影中心又称为**视点**，它相当于观察者的**眼睛**。投影面置于视点与物体之间，将物体上的各点与视点相连所得的**投影线与投影面的交点**就是三维物体上相应点的透视变换结果。



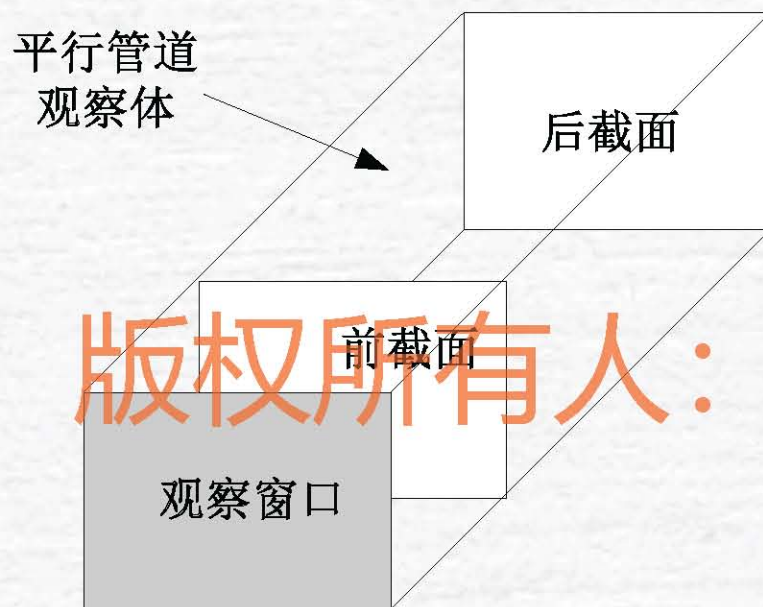
4.2.3 三维观察变换

4.2.3.1 观察空间

定义了观察窗口的大小，可以利用窗口边界来定义观察空间。只有在观察空间中的物体才会在输出设备中显示，而其它的景物将被裁剪掉。



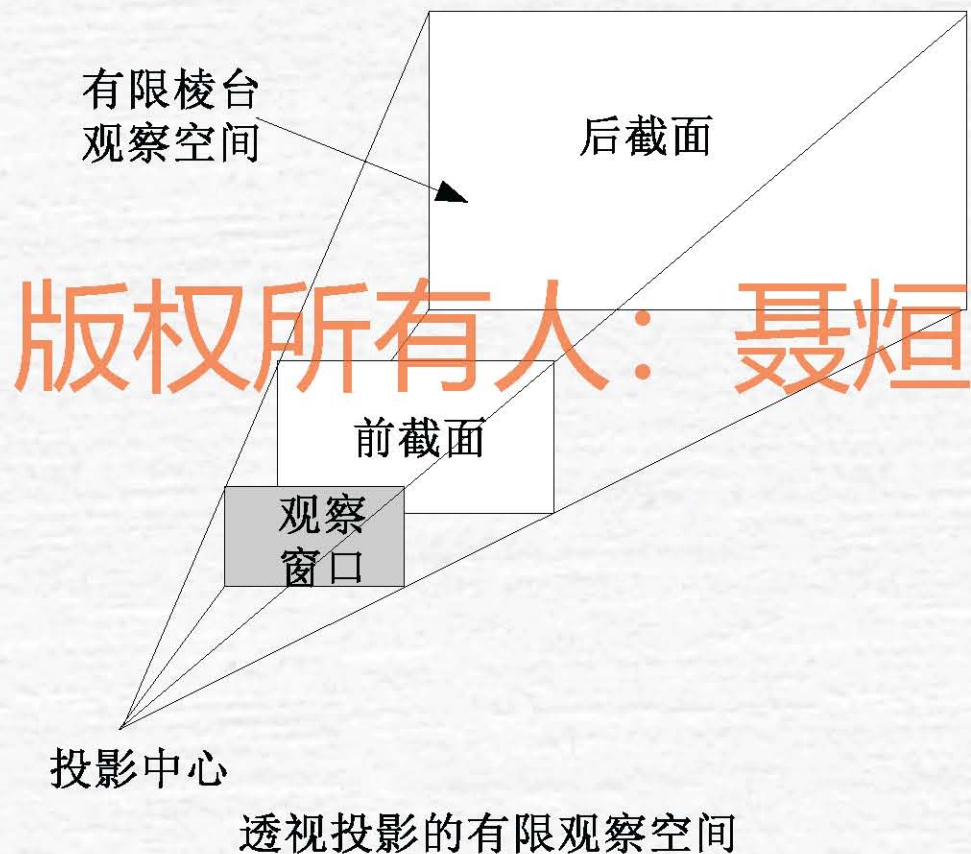
4.2.3.1 观察空间(续)



正投影的有限观察空间

4.2.3 三维观察变换(续)

4.2.3.1 观察空间



4.2.3 三维观察变换(续)

4.2.3.2 三维观察流程



三维观察流程

4.2.4 OpenGL中的三维图形变换

变换是图形制作和图形显示中的重要内容，OpenGL中的函数提供了丰富的图形变换功能。根据变换的不同性质，OpenGL中的图形变换分为四类：视点变换、模型变换、投影变换、视区变换。这些图形变换函数都是通过矩阵操作来实现的。

1. 模型变换
2. 视点变换
3. 投影变换
4. 视区变换
5. 裁剪变换

版权所有人：聂烜

在OpenGL编程过程中，坐标变换是一个贯穿始终的操作。程序员必须在头脑中对整个坐标变换过程有一个清晰的想象，才能将所建的场景模型正确地显示在屏幕上。

OpenGL的坐标变换过程类似于用照相机拍摄照片的过程。使用照相机的步骤如下：

- 1). 竖起三角架，将照相机对准场景（视图变换）。
- 2). 将要拍的场景置于所要求的位置上。（模型变换）。
- 3). 选择照相机透镜或调整焦距（投影变换）。
- 4). 确定最终的照片需要多大。例如，放大照片（视口变换）。

第一步, 指定视图和造型变换, 两者结合称为视图造型变换。这个变换将输入的顶点从世界坐标变换到视觉坐标。

第二步, 指定投影变换, 使对象从视觉坐标变换到裁剪坐标。这个变换定义一个视图体 (viewing volume), 在视图体外的对象被裁剪掉。

第三步, 通过对坐标值除以 w , 执行透视除法, 变换到规格化设备坐标。

第四步, 通过视口变换将坐标变换到窗口坐标。

1、OpenGL中的矩阵操作函数

✓指定特定的矩阵作为当前矩阵：

```
void glLoadMatrix{fd} (const TYPE *m)
```

参数说明：m为一个单精度或双精度浮点数指针，指向一个按列存储的4×4的矩阵。

这里，当前矩阵指经过一系列变化后的用户坐标

例如：glFloat m[]={a1, a2, a3, ..., a14, a15, a16}

```
glLoadMatrix(m);
```

以上程序将栈顶矩阵设置为：

$$T_3 = \begin{bmatrix} a1 & a5 & a9 & a13 \\ a2 & a6 & a10 & a14 \\ a3 & a7 & a11 & a15 \\ a4 & a8 & a12 & a16 \end{bmatrix}$$

进行各种变换
需要首先了解
OpenGL的矩
阵操作

✓ 指定当前矩阵为单位阵：

glLoadIdentity();

modelview 模型观察矩阵，用来表示物体的位置变化和观察点的改变

OpenGL中对矩阵的操作是有累加作用的

当画了若干图形，同时也移动了多次坐标系后，又想在世界坐标的原点画东西时，应使用该函数

用单位矩阵代替当前矩阵就是使自己对坐标的变化无效，一切从头开始。

1、OpenGL中的矩阵操作函数(续)

当前矩阵函数 `glMatrixMode(GLenum mode)`

功能: `mode`用于指定哪一种矩阵栈是其后矩阵操作的目标。`mode`可取:

`GL_MODELVIEW`:把其后的矩阵操作施加于造型视图矩阵栈。(默认)

`GL_PROJECTION`:把其后的矩阵操作施加于投影矩阵栈。

`GL_TEXTURE`: 把其后的矩阵操作施加于纹理矩阵栈。

矩阵相乘的函数: `void glMultMatrix{fd}(const TYPE*m)`

功能: 用当前矩阵乘以这个函数所提供的矩阵, 并把结果作为当前矩阵。

`m`为一个单精度或双精度浮点数指针, 指向一个按列存储的 4×4 的矩阵。

1、OpenGL中的矩阵操作函数(续)

✓入栈函数:

```
void glPushMatrix(void)
```

功能: 把当前的矩阵拷贝到栈中。

✓出栈函数:

```
void glPopMatrix(void)
```

最后压入栈的矩阵恢复为当前矩阵。

作用: 保护当前矩阵现场。

例如: 当做了一些移动或旋转等变换后, 使用glPushMatrix();

OpenGL 会把变换后的位置和角度保存起来。

然后再随便做第二次移动或旋转变换时, 再用glPopMatrix();

OpenGL 就把刚刚保存的那个位置和角度恢复。

2、模型变换

OpenGL中有三个用于模型变换的函数，
`glTranslate*()`、`glRotate*()`和`glScale*()`，
它们分别通过平移、旋转和缩放来操作变换一个指定的对象。


```
void glTranslate*(TYPE x, TYPE y, TYPE z);
```

该函数以平移矩阵乘当前矩阵。

x, y, z 指定沿世界坐标系 x 、 y 、 z 轴的平移量。

```
void glRotate*(TYPE angle, TYPE x, TYPE y, TYPE z);
```

该函数以旋转矩阵乘当前矩阵，其中：

$angle$ 指定旋转的角度（以度为单位）。

x, y, z 指定旋转轴向量的三个分量(该向量位于世界坐标系中)。

```
void glScale*(TYPE x, TYPE y, TYPE z);
```

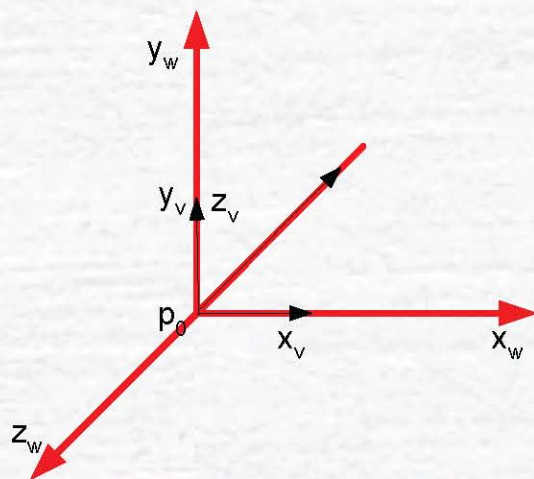
该函数以缩放矩阵乘当前矩阵， x 、 y 、 z 指定沿 x 、 y 和 z 轴的比例因子。

例

3、视点变换

世界坐标系，也称为全局坐标系。它是一个右手坐标系，可以认为该坐标系是固定不变的，在初始态下，其 x 轴为沿屏幕水平向右， y 轴为沿屏幕垂直向上， z 轴则为垂直屏幕面向外指向用户，当然，如果在程序中对视点进行了转换，就不能再认为是这样的了。

视觉坐标系，也称为局部坐标系。它是一个左手坐标系，该坐标系是可以活动的。在初始态下，其原点及 x, y 轴分别与世界坐标系的原点及 x, y 轴重合，而 z 轴则正好相反，即为垂直屏幕面向内。



➤视点变换也可以称为视图变换，是指改变对象观察点的位置和方向。类似放置照相机的三角架，通过对相机的平移和旋转，把相机指向拍摄对象。

➤视图变换改变视点的位置和方向，也就是改变视觉坐标系。 版权所有人： 聂烜

➤在世界坐标系中，视点和物体的位置是一个相对的关系，对物体作一些平移、旋转变换，必定可以通过对视点作相应的平移、旋转变换来达到相同的视觉效果。

完成视图变换可以有以下几种方法：

1. 利用一个或几个造型变换命令(即`glTranslate*()`和`glRotate*()`)。由于这些命令是在`GL_MODELVIEW`状态下执行的，所以较难和那些造型变换命令区分开来，移动视点的变换和移动物体的变换很容易混淆。为了便于建立清晰的物体和场景模型，可以认为只有其中一个变换在起作用，比如认为只有模型变换的话，那么`glTranslate*()`和`glRotate*()`将统一被视为对物体的变换。

2. 利用实用库函数gluLookAt()设置视觉坐标系。在实际的编程应用中，用户在完成场景的建模后，往往需要选择一个合适的视角或者不停地变换视角，以对场景作观察。实用库函数gluLookAt()就提供了这样的一个功能。

版权所有人：聂烜

```
void gluLookAt(GLdouble eyex, GLdouble eyey, GLdouble eyez,  
               GLdouble centerx, GLdouble centery, GLdouble centerz,  
               GLdouble upx, GLdouble upy, GLdouble upz);
```

该函数定义一个视图矩阵。

eyex, eyey, eyez 指定视点的位置

centerx, centery, centerz 指定参考点的位置

upx, upy, upz 指定视点向上的方向

3. 创建封装旋转和平移命令的实用函数。有些应用需要用简便方法指定视图变换的定制函数。

例如，在飞机飞行中指定滚动、俯仰和航向旋转角，或对环绕对象运动的照相机指定一种利用极坐标的变换。

版权所有人： 聂烜

4.2.4 OpenGL中的三维图形变换(续)

4、投影变换

投影变换就是要确定一个取景体积（空间），其作用有两个：

版权所有人：聂烜

1). 确定物体投影到屏幕的方式，即是透视投影还是正交投影。

2). 确定从图象上裁剪掉哪些物体或物体的某些部分。
投影变换包括透视投影和正交投影（平行投影）。

1. 透视投影

透视投影的示意图如下，其取景体积是一个截头锥体，在这个体积内的物体投影到锥的顶点，用 `glFrustum()` 函数定义这个截头锥体，这个取景体积可以是不对称的，计算透视投影矩阵 M ，并乘以当前矩阵 C ，使 $C=CM$ 。

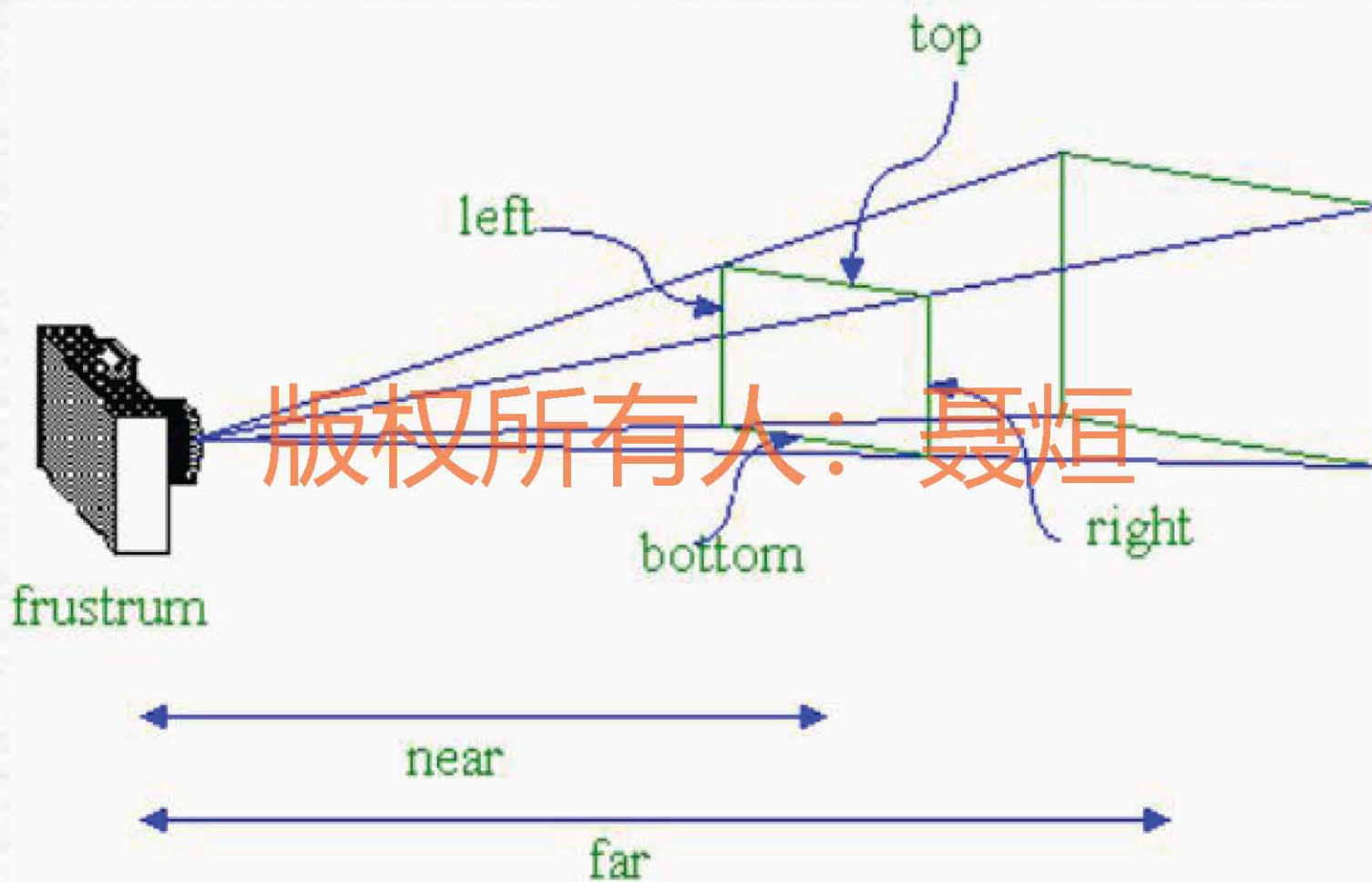
```
void glFrustum(GLdouble left, GLdouble  
               right, GLdouble bottom, GLdouble top, GLdouble  
               near, GLdouble far);
```

该函数以透视矩阵乘当前矩阵

`left, right` 指定左右垂直裁剪面的坐标。

`bottom, top` 指定底和顶水平裁剪面的坐标。

`near, far` 指定近和远深度裁剪面的距离，两个距离一定是正的

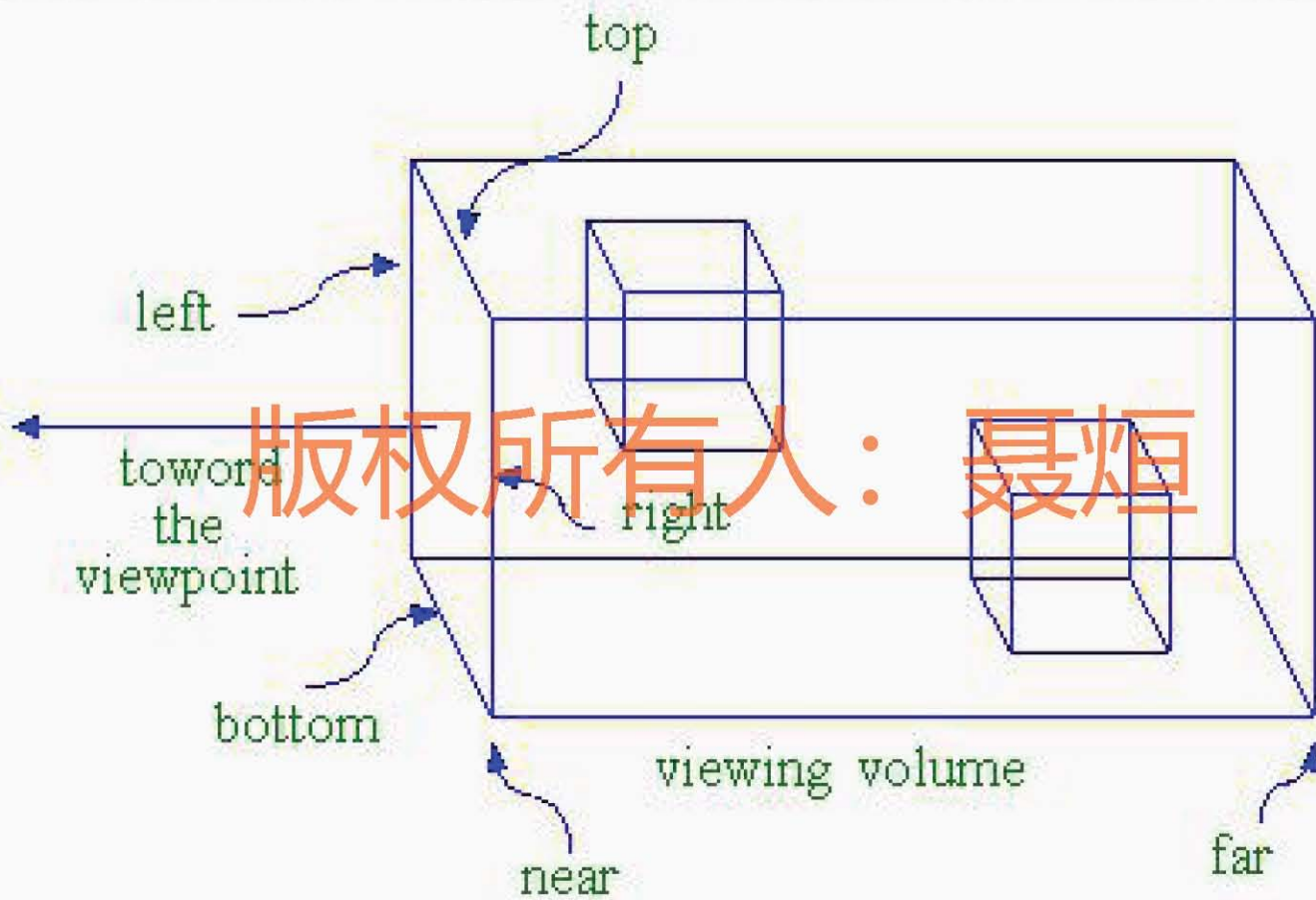


2. 正交投影

正交投影的示意图见下：其取景体积是一个各面均为矩形的六面体，用`glOrtho()`函数创建正交平行的取景体积，计算正交平行取景体积矩阵 M ，并乘以当前矩阵 C ，使 $C=CM$ 。

```
void glOrtho(Gldouble left, Gldouble  
right, Gldouble bottom, Gldouble top, Gldouble  
near, Gldouble far);
```

该函数以正交投影矩阵乘当前矩阵。



版权所有人：聂烜

4.2.4 OpenGL中的三维图形变换(续)

5. 视区变换

视口就是窗口中矩形绘图区。用窗口管理器在屏幕上打开一个窗口时，已经自动地把视口区设为整个窗口的大小，可以用glViewport命令设定一个较小的绘图区，利用这个命令还可以在同一个窗口上同时显示多个视图，达到分屏显示的目的。

```
void glViewport(Glint x, Glint y, Glsize  
width, Glsize height);
```

该函数设置视口的大小。

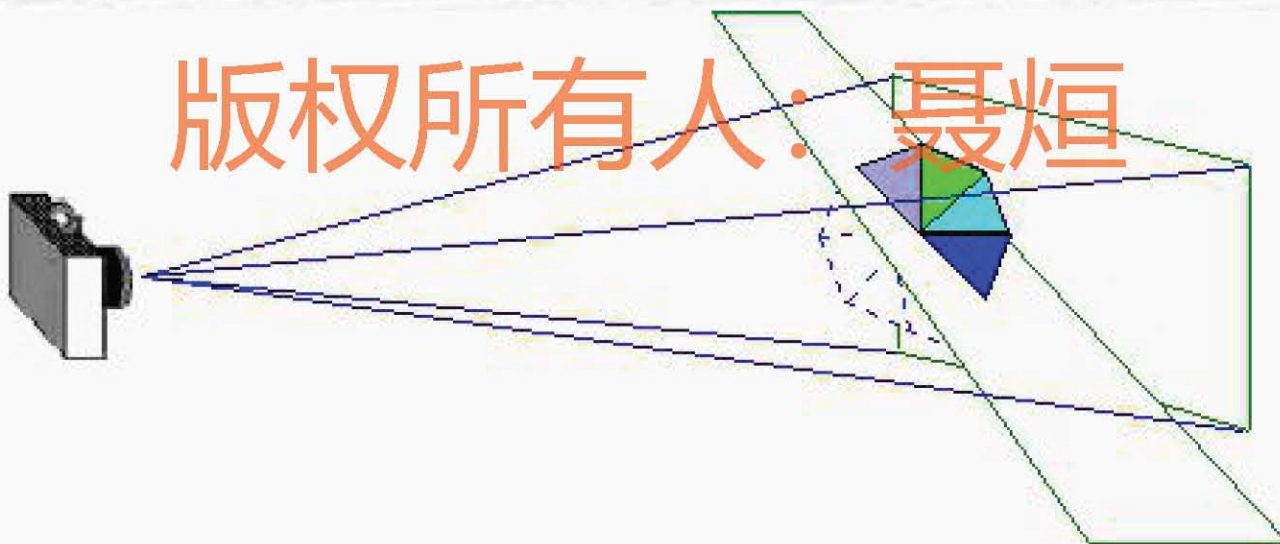
x, y 指定视口矩形的左下角坐标（以像素为单位），缺省值为（0,0）。

width, height 分别指定视口的宽和高。

4.2.4 OpenGL 中的三维图形变换(续)

5、裁剪变换

除了视图体的六个裁剪面（左、右、底、顶、近和远）外，还能定义最多六个附加的裁剪面来进一步限制视图体，如下图所示。



附加裁剪面可用于显示物体的剖面图，每个裁剪面通过指定方程 $Ax+By+Cz+D=0$ 中的系数来确定。裁剪体进行造型和视图变换时，裁剪面自动进行相应的变换。最后的裁剪体成为视图体和附加裁剪面所定义的全部半空间的交。

`void glClipPlane(Glenum plane,`
`const Gldouble *equation);`

该函数定义附加裁剪面，其中：

`plane` 用符号名`GL_CLIP_PLANEi`指定裁剪面，其中`i`为0和5之间的整数，指定六个裁剪面中一个。

`equation` 指定四个值的数值，存放平面方程的四个参数。

在定义每个附加裁剪面之前，必须发出激活命令：

```
glEnable(GL_CLIP_PLANEi);
```

用如下命令可去活一个平面：

```
glDisable(GL_CLIP_PLANEi);
```

有些OpenGL允许设置的裁剪面个数多于六个，可利用下面命令来查询支持裁剪面的数目：

```
glGetIntegerv(GL_MAX_CLIP_PLANEi, GLint * p);
```

该函数返回后，参数指针p所指向的整数值即为该系统所支持裁剪面的数目。

4.3 实例分析

- 例1 **Opengl**程序基本架构
- 例2 基于**MFC**的**Opengl**程序（步骤）
- 例3 彩色立方体动画
- 例4 被剪切的茶壶

版权所有：聂烜